Markov Decision Process (MDP)

Implementation, Execution & Output Report

All Python Modules – Source Code & Console Output

Table of Contents

1. Module 1 – Define MDP States, Actions, Transition Probabilities & Rewards

Source Code

```
states = ["S1", "S2", "S3"]
actions = ["A1", "A2"]

num_states = len(states)
num_actions = len(actions)

transition_probabilities = {
    ("S1", "A1"): {"S2": 0.6, "S3": 0.2},
    ("S1", "A2"): {"S2": 0.4, "S3": 0.8},

    ("S2", "A1"): {"S1": 0.7, "S3": 0.5},
    ("S2", "A2"): {"S1": 0.3, "S3": 0.5},

    ("S3", "A1"): {"S1": 0.9, "S2": 0.4},
    ("S3", "A2"): {"S1": 0.1, "S2": 0.6}
}

rewards = {
    ("S1", "A1", "S2"): 5,
    ("S1", "A2", "S2"): 10,
    ("S1", "A1", "S3"): -1,
    ("S1", "A2", "S3"): -5,

    ("S2", "A1", "S1"): 3,
    ("S2", "A2", "S1"): 7,
    ("S2", "A1", "S3"): 2,
    ("S2", "A2", "S3"): 1,

    ("S3", "A1", "S1"): 4,
    ("S3", "A2", "S1"): 6,
    ("S3", "A1", "S2"): 0,
    ("S3", "A2", "S2"): -2
}

print("Number of States:", num_states)
print("States:", states)

print("\nNumber of Actions:", num_actions)
print("Actions:", actions)

print("\nTransition Probabilities")
for key, value in transition_probabilities.items():
    state, action = key
    print(f"{state}, {action} -> {value}")

print("\nRewards")
for key, value in rewards.items():
    state, action, next_state = key
    print(f"R({state}, {action}, {next_state}) = {value}")
```

Console Output

Module1.py - Output

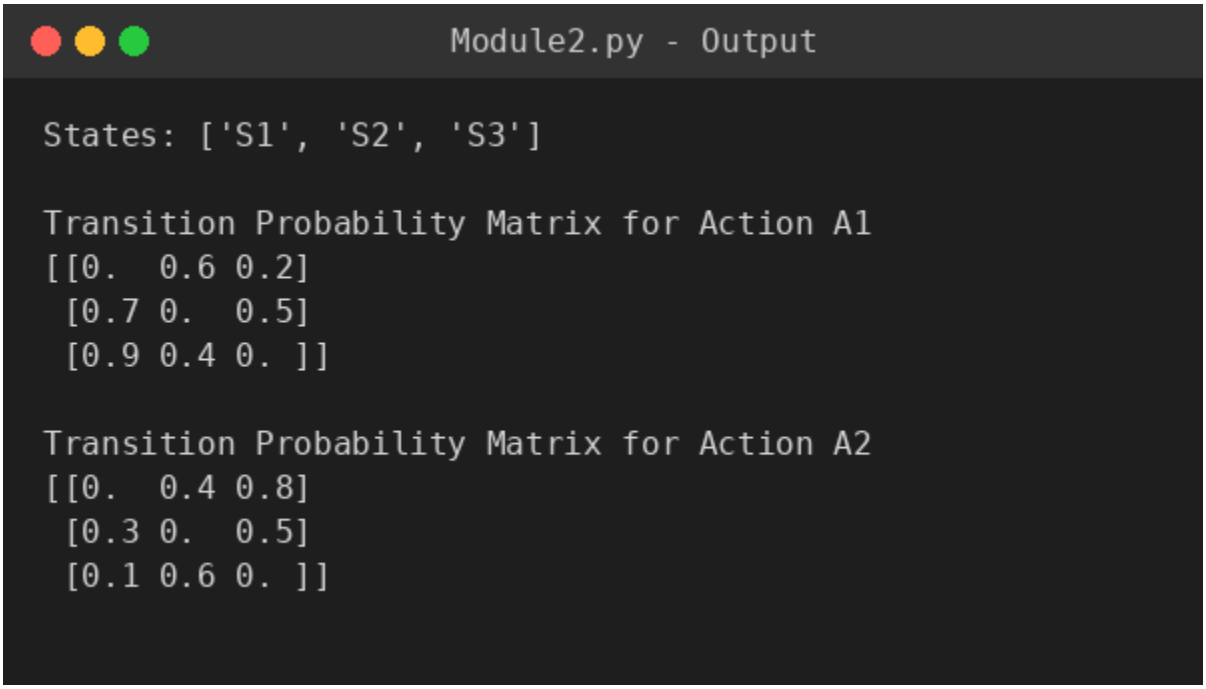
```
Number of States: 3
States: ['S1', 'S2', 'S3']

Number of Actions: 2
Actions: ['A1', 'A2']

Transition Probabilities
S1, A1 -> {'S2': 0.6, 'S3': 0.2}
S1, A2 -> {'S2': 0.4, 'S3': 0.8}
S2, A1 -> {'S1': 0.7, 'S3': 0.5}
S2, A2 -> {'S1': 0.3, 'S3': 0.5}
S3, A1 -> {'S1': 0.9, 'S2': 0.4}
S3, A2 -> {'S1': 0.1, 'S2': 0.6}

Rewards
R(S1, A1, S2) = 5
R(S1, A2, S2) = 10
R(S1, A1, S3) = -1
R(S1, A2, S3) = -5
R(S2, A1, S1) = 3
R(S2, A2, S1) = 7
R(S2, A1, S3) = 2
R(S2, A2, S3) = 1
R(S3, A1, S1) = 4
R(S3, A2, S1) = 6
R(S3, A1, S2) = 0
R(S3, A2, S2) = -2
```

2. Module 2 – Transition Probability Matrices (NumPy)

Source Code

```
import numpy as np

states = ["S1", "S2", "S3"]

P_A1 = np.array([
    [0.0, 0.6, 0.2],
    [0.7, 0.0, 0.5],
    [0.9, 0.4, 0.0]
])

P_A2 = np.array([
    [0.0, 0.4, 0.8],
    [0.3, 0.0, 0.5],
    [0.1, 0.6, 0.0]
])

print("States:", states)

print("\nTransition Probability Matrix for Action A1")
print(P_A1)

print("\nTransition Probability Matrix for Action A2")
print(P_A2)
```

Console Output

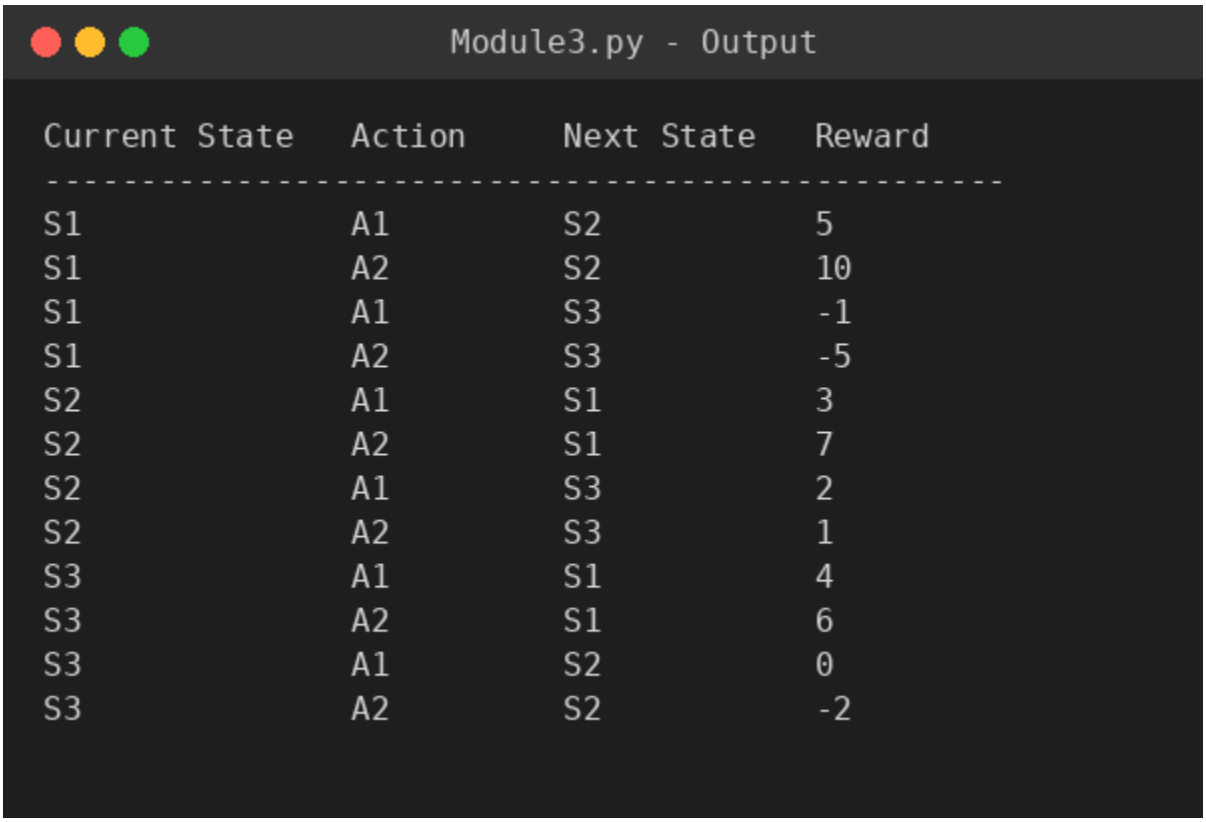


3. Module 3 – Reward Table (Pandas Style Print)

Source Code

```
import pandas as pd

reward_data = [
    ["S1", "A1", "S2", 5],
    ["S1", "A2", "S2", 10],
    ["S1", "A1", "S3", -1],
    ["S1", "A2", "S3", -5],
    ["S2", "A1", "S1", 3],
    ["S2", "A2", "S1", 7],
    ["S2", "A1", "S3", 2],
    ["S2", "A2", "S3", 1],
    ["S3", "A1", "S1", 4],
    ["S3", "A2", "S1", 6],
    ["S3", "A1", "S2", 0],
    ["S3", "A2", "S2", -2]
]

print("{:<15} {:<10} {:<12} {:<8}".format(
    "Current State", "Action", "Next State", "Reward"))
print("-" * 50)

for row in reward_data:
    print("{:<15} {:<10} {:<12} {:<8}".format(*row))
```

Console Output



4. Module 4 – Expected Reward Calculation

Source Code

```
states = ["S1", "S2", "S3"]
actions = ["A1", "A2"]

transition_probabilities = {
    ("S1", "A1"): {"S2": 0.6, "S3": 0.2},
    ("S1", "A2"): {"S2": 0.4, "S3": 0.8},

    ("S2", "A1"): {"S1": 0.7, "S3": 0.5},
    ("S2", "A2"): {"S1": 0.3, "S3": 0.5},

    ("S3", "A1"): {"S1": 0.9, "S2": 0.4},
    ("S3", "A2"): {"S1": 0.1, "S2": 0.6}
}

rewards = {
    ("S1", "A1", "S2"): 5,
    ("S1", "A1", "S3"): -1,
    ("S1", "A2", "S2"): 10,
    ("S1", "A2", "S3"): -5,

    ("S2", "A1", "S1"): 3,
    ("S2", "A1", "S3"): 2,
    ("S2", "A2", "S1"): 7,
    ("S2", "A2", "S3"): 1,

    ("S3", "A1", "S1"): 4,
    ("S3", "A1", "S2"): 0,
    ("S3", "A2", "S1"): 6,
    ("S3", "A2", "S2"): -2
}

print("Expected Reward Calculation\n")

for state in states:
    for action in actions:

        print(f"State = {state}, Action = {action}")

        expected_reward = 0

        for next_state, probability in transition_probabilities[(state, action)].items():

            reward = rewards[(state, action, next_state)]
            value = probability * reward

            print(f"P({next_state}|{state},{action}) × R({state},{action},{next_state})")
            print(f"= {probability} × {reward} = {value}")

            expected_reward += value

        print(f"Expected Reward ({state}, {action}) = {expected_reward}")
        print("-" * 50)
```

Console Output

Module4.py - Output

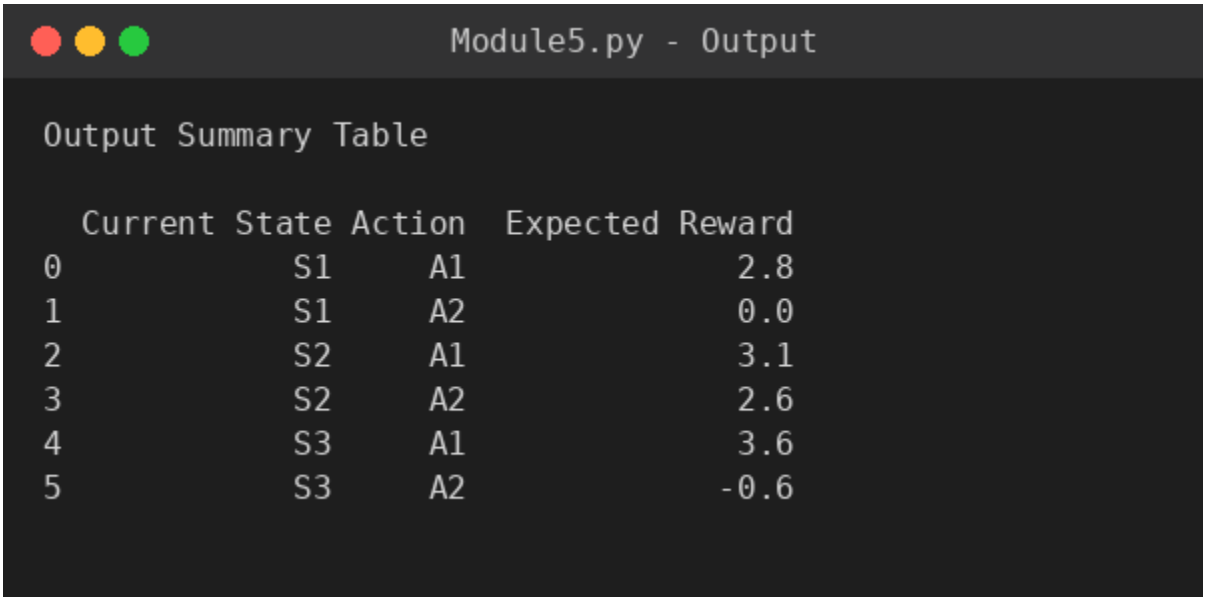
```
Expected Reward Calculation

State = S1, Action = A1
P(S2|S1,A1) × R(S1,A1,S2)
= 0.6 × 5 = 3.0
P(S3|S1,A1) × R(S1,A1,S3)
= 0.2 × -1 = -0.2
Expected Reward (S1, A1) = 2.8
------------------------------------------------
State = S1, Action = A2
P(S2|S1,A2) × R(S1,A2,S2)
= 0.4 × 10 = 4.0
P(S3|S1,A2) × R(S1,A2,S3)
= 0.8 × -5 = -4.0
Expected Reward (S1, A2) = 0.0
------------------------------------------------
State = S2, Action = A1
P(S1|S2,A1) × R(S2,A1,S1)
= 0.7 × 3 = 2.0999999999999996
P(S3|S2,A1) × R(S2,A1,S3)
= 0.5 × 2 = 1.0
Expected Reward (S2, A1) = 3.0999999999999996
------------------------------------------------
State = S2, Action = A2
P(S1|S2,A2) × R(S2,A2,S1)
= 0.3 × 7 = 2.1
P(S3|S2,A2) × R(S2,A2,S3)
= 0.5 × 1 = 0.5
Expected Reward (S2, A2) = 2.6
------------------------------------------------
State = S3, Action = A1
P(S1|S3,A1) × R(S3,A1,S1)
= 0.9 × 4 = 3.6
P(S2|S3,A1) × R(S3,A1,S2)
= 0.4 × 0 = 0.0
Expected Reward (S3, A1) = 3.6
------------------------------------------------
State = S3, Action = A2
P(S1|S3,A2) × R(S3,A2,S1)
= 0.1 × 6 = 0.6000000000000001
P(S2|S3,A2) × R(S3,A2,S2)
= 0.6 × -2 = -1.2
```

5. Module 5 – Output Summary Table (Pandas DataFrame)

Source Code

```
import pandas as pd

summary_data = [
    ["S1", "A1", 2.8],
    ["S1", "A2", 0.0],
    ["S2", "A1", 3.1],
    ["S2", "A2", 2.6],
    ["S3", "A1", 3.6],
    ["S3", "A2", -0.6]
]

summary_table = pd.DataFrame(
    summary_data,
    columns=["Current State", "Action", "Expected Reward"]
)

print("Output Summary Table\n")
print(summary_table)
```

Console Output



6. Module 6 – MDP Activity Diagram Generation (Graphviz)

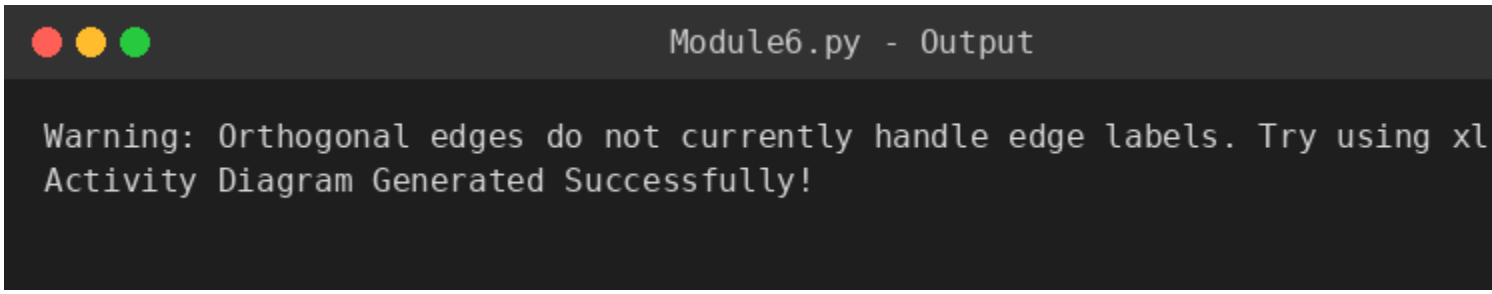
Source Code

```
from graphviz import Digraph

activity = Digraph("MDP_Activity_Diagram", format="png")

# Top to Bottom Layout
activity.attr(rankdir="TB", bgcolor="white", splines="ortho", nodesep="0.6", ranksep="0.8")

# Default Node Style
activity.attr("node",
              fontname="Arial",
              fontsize="12",
              style="filled",
              color="black")

# ----------------------------
# Nodes
# ----------------------------

activity.node("Start",
              "START",
              shape="circle",
              fillcolor="#90EE90")

activity.node("Init",
              "Initialize\nMDP",
              shape="box",
              fillcolor="#87CEFA")

activity.node("State",
              "Define\nStates\n(S1,S2,S3)",
              shape="box",
              fillcolor="#FFFACD")

activity.node("Action",
              "Define\nActions\n(A1,A2)",
              shape="box",
              fillcolor="#FFFACD")

activity.node("TP",
              "Store Transition\nProbabilities",
              shape="box",
              fillcolor="#D8BFD8")

activity.node("Reward",
              "Store\nReward Matrix",
              shape="box",
              fillcolor="#F4A460")

activity.node("Display",
              "Display States,\nActions,\nMatrices",
              shape="box",
              fillcolor="#AFEEEE")

activity.node("Decision",
              "More\nStates?",
              shape="diamond",
              fillcolor="#FFD700")
```

```
activity.node("Current",
              "Select\nCurrent State",
              shape="box",
              fillcolor="#E6E6FA")

activity.node("Choose",
              "Choose\nAction",
              shape="box",
              fillcolor="#E6E6FA")

activity.node("Transition",
              "Apply\nTransition\nProbability",
              shape="box",
              fillcolor="#98FB98")

activity.node("Next",
              "Move to\nNext State",
              shape="box",
              fillcolor="#98FB98")

activity.node("RewardCalc",
              "Receive\nReward",
              shape="box",
              fillcolor="#FFB6C1")

activity.node("Expected",
              "Calculate\nExpected Reward",
              shape="box",
              fillcolor="#87CEEB")

activity.node("Summary",
              "Generate\nSummary Table",
              shape="box",
              fillcolor="#FFA07A")

activity.node("Visual",
              "Visualize\nGrid Environment\n& State Diagram",
              shape="box",
              fillcolor="#ADD8E6")

activity.node("End",
              "END",
              shape="doublecircle",
              fillcolor="#90EE90")

# ----------------------------
# Connections
# ----------------------------

activity.edge("Start", "Init")
activity.edge("Init", "State")
activity.edge("State", "Action")
activity.edge("Action", "TP")
activity.edge("TP", "Reward")
activity.edge("Reward", "Display")
activity.edge("Display", "Decision")

activity.edge("Decision", "Current", label="Yes")
activity.edge("Current", "Choose")
activity.edge("Choose", "Transition")
activity.edge("Transition", "Next")
```

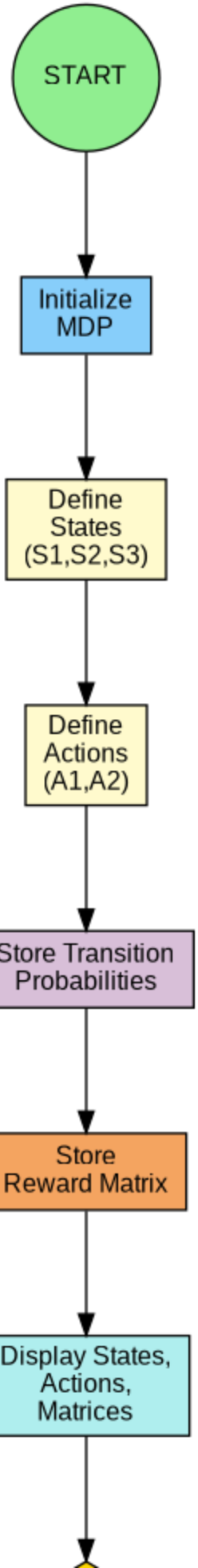
```
activity.edge("Next", "RewardCalc")
activity.edge("RewardCalc", "Expected")
activity.edge("Expected", "Decision")

activity.edge("Decision", "Summary", label="No")
activity.edge("Summary", "Visual")
activity.edge("Visual", "End")

# ----------------------------
# Generate Diagram
# ----------------------------

activity.render("MDP_Activity_Diagram", view=True)

print("Activity Diagram Generated Successfully!")
```

Console Output



7. MDP Activity Diagram (Full Flowchart)

The diagram below was generated by Module 6 using Graphviz and represents the complete activity flow of the MDP program, from initialization through state/action definition, transition & reward storage, iterative processing, expected reward calculation, and final summary/visualization.


START
Initialize MDP
Define States (S1,S2,S3)
Define Actions (A1,A2)
Store Transition Probabilities
Store Reward Matrix
Display States, Actions, Matrices